

Samostalna vježba — Event Registration forma s validacijom

Naziv projekta

08_React/13_Practice_EventRegistrationForm/

Cilj vježbe

U ovoj vježbi trebate samostalno izraditi kontroliranu React formu za prijavu na događaj.

Zamislite da izrađujete malu formu za prijavu na frontend radionicu. Korisnik treba upisati svoje podatke, odabrati vrstu događaja, napisati kratku motivaciju i prihvatiti pravila sudjelovanja.

Forma mora biti kontrolirana, što znači da se sve vrijednosti forme moraju čuvati u React stateu.

Osim toga, forma mora imati validaciju. Ako korisnik pokuša poslati formu s neispravnim podacima, trebaju se prikazati poruke greške.

Scenarij

Izrađujete formu za događaj:

Frontend Workshop Registration

Korisnik se želi prijaviti na jednu od radionica:

HTML & CSS Workshop
JavaScript Basics Workshop
React Beginner Workshop
React Forms Workshop

Forma treba prikupiti osnovne podatke i provjeriti jesu li ispravno uneseni.

Što trebate napraviti

Napravite React aplikaciju s jednom formom.

Forma mora imati sljedeća polja:

Full name

Email

Age

Workshop

Experience level

Motivation message

Accept rules checkbox

Nakon uspješnog slanja forme, aplikacija treba prikazati poruku uspjeha i kratki sažetak prijave.

Struktura projekta

Možete sve napraviti u `App.jsx`.

Za ovu vježbu nije potrebno razdvajati formu u više komponenti.

Predložena struktura:

src/

App.jsx

App.css

main.jsx

Cilj ove vježbe nije organizacija komponenti, nego razumijevanje kontrolirane forme i validacije.

Korak 1 — Kreirajte projekt

Napravite novi Vite React projekt:

```
npm create vite@latest event-registration-form -- --template react
```

Uđite u projekt:

```
cd event-registration-form
```

Instalirajte dependencyje:

npm install

Pokrenite aplikaciju:

npm run dev

Korak 2 — Pripremite osnovni layout

U `App.jsx` napravite osnovnu strukturu stranice.

Stranica treba imati:

naslov
kratki opis
formu

Primjer sadržaja:

Frontend Workshop Registration

Fill out the form below to register for one of our upcoming frontend workshops.

Forma treba biti smještena u karticu ili centralni container.

Korak 3 — Napravite početni `formData` state

Napravite jedan state objekt koji čuva sve vrijednosti forme.

State treba sadržavati:

fullName
email
age
workshop
experienceLevel
motivation
acceptedRules

Početne vrijednosti trebaju biti prazni stringovi, osim checkboxa koji treba biti `false`.

Primjer oblika statea:

```
const [formData, setFormData] = useState({
  fullName: "",
  email: "",
  age: "",
  workshop: "",
  experienceLevel: "",
  motivation: "",
  acceptedRules: false
});
```

Korak 4 — Napravite **errors state**

Napravite poseban state za greške.

Treba imati ista polja kao **formData**, ali vrijednosti trebaju biti poruke greške.

Početno sve vrijednosti trebaju biti prazni stringovi.

Primjer:

```
const [errors, setErrors] = useState({
  fullName: "",
  email: "",
  age: "",
  workshop: "",
  experienceLevel: "",
  motivation: "",
  acceptedRules: ""
});
```

Korak 5 — Dodajte sva polja u formu

Forma mora sadržavati sljedeća polja.

Full name

Običan tekstualni input.

`type="text"`

Mora biti kontroliran pomoću:

value
onChange

Email

Email input.

type="email"

Mora biti kontroliran.

Age

Number input.

type="number"

Mora biti kontroliran.

Iako je input tipa `number`, vrijednost u React stateu će i dalje dolaziti kao string. To je u redu za ovu vježbu, ali kod validacije ćete po potrebi koristiti `Number(...)`.

Workshop

`select` element.

Opcije:

Choose a workshop
HTML & CSS Workshop
JavaScript Basics Workshop
React Beginner Workshop
React Forms Workshop

Početna opcija treba imati prazan value:

```
<option value="">Choose a workshop</option>
```

Experience level

`select` element.

Opcije:

Choose your level

Beginner

Some experience

Intermediate

Početna opcija treba imati prazan value.

Motivation message

`textarea`.

Korisnik treba napisati zašto želi sudjelovati na radionici.

Accept rules

Checkbox.

Tekst:

I accept the workshop rules.

Checkbox mora koristiti:

`checked={formData.acceptedRules}`

a ne `value`.

Korak 6 — Napravite `handleChange`

Napravite jednu `handleChange` funkciju koja radi za sva polja.

Funkcija mora moći raditi i s običnim inputima i s checkboxom.

Treba koristiti:

```
const { name, value, type, checked } = event.target;
```

Ako je polje checkbox, u state treba spremiti `checked`.

Ako nije checkbox, u state treba spremiti `value`.

Logika treba biti:

ako je type checkbox → spremi checked
inače → spremi value

Sva polja u formi moraju imati odgovarajući `name` atribut koji odgovara ključu u `formData` objektu.

Na primjer:

```
<input name="fullName" />
```

mora ažurirati:

```
formData.fullName
```

Korak 7 — Napravite `validateField`

Napravite funkciju:

```
function validateField(name, value) {  
  // validation logic  
}
```

Ova funkcija treba validirati jedno polje i vratiti poruku greške.

Ako je polje ispravno, treba vratiti prazan string:

```
return "";
```

Pravila validacije

Full name

Pravila:

obavezno
mora imati barem 3 znaka

Poruke greške mogu biti:

Full name is required.

Full name must have at least 3 characters.

Email

Pravila:

obavezno
mora sadržavati @

Poruke:

Email is required.
Email must contain @.

Age

Pravila:

obavezno
mora biti broj
mora biti najmanje 16
mora biti najviše 80

Poruke:

Age is required.
Age must be a number.
You must be at least 16 years old.
Age cannot be greater than 80.

Napomena: vrijednost iz inputa je string, zato za provjeru koristite `Number(value)`.

Workshop

Pravila:

obavezno odabrati radionicu

Poruka:

Please choose a workshop.

Experience level

Pravila:

obavezno odabrati experience level

Poruka:

Please choose your experience level.

Motivation message

Pravila:

obavezno
mora imati barem 30 znakova

Poruke:

Motivation message is required.
Motivation message must have at least 30 characters.

Accept rules

Pravila:

mora biti označeno

Poruka:

You must accept the workshop rules.

Korak 8 — Validirajte polje tijekom promjene

Kada korisnik mijenja bilo koje polje, trebate:

ažurirati formData
validirati to polje
ažurirati errors za to polje

Drugim riječima, `handleChange` treba raditi dvije stvari:

1. spremi novu vrijednost u formData
2. spremi rezultat validateField u errors

Primjer ponašanja:

Ako korisnik u `Full name` upiše samo:

An

ispod polja treba se prikazati:

Full name must have at least 3 characters.

Ako zatim upiše:

Ana

greška treba nestati.

Korak 9 — Napravite `validateForm`

Napravite funkciju:

```
function validateForm() {  
  // validate all fields  
}
```

Ona treba validirati sva polja i vratiti novi objekt grešaka.

Može koristiti `validateField` za svako polje.

Primjer ideje:

```
const newErrors = {  
  fullName: validateField("fullName", formData.fullName),  
  email: validateField("email", formData.email),  
  age: validateField("age", formData.age),  
  workshop: validateField("workshop", formData.workshop),  
  experienceLevel: validateField(  
    "experienceLevel",  
    formData.experienceLevel  
  ),  
  motivation: validateField("motivation", formData.motivation),  
  acceptedRules: validateField(  
    "acceptedRules",  
    formData.acceptedRules  
  )  
}
```

```
)  
};
```

Korak 10 — Napravite `handleSubmit`

Forma mora imati:

```
onSubmit={handleSubmit}
```

U `handleSubmit` funkciji trebate:

- spriječiti defaultno ponašanje forme
- validirati cijelu formu
- spremiti greške u errors state
- provjeriti postoji li barem jedna greška
- ako postoji greška, prekinuti submit
- ako nema grešaka, prikazati success rezultat

Za provjeru ima li grešaka možete koristiti:

```
Object.values(validationErrors).some(error => error !== "")
```

Korak 11 — Prikažite poruke greške

Ispod svakog polja prikažite poruku greške ako postoji.

Primjer:

```
{errors.fullName && (  
  <p className="error-message">{errors.fullName}</p>  
)}
```

Ovo treba napraviti za sva polja.

Korak 12 — Prikažite success poruku

Nakon uspješnog submitanja prikažite poruku:

Registration successful!

Ispod toga prikazite kratki sažetak prijave.

Sažetak treba sadržavati:

Full name

Email

Selected workshop

Experience level

Primjer:

Thank you, Ana Horvat.

You registered for React Beginner Workshop.

Experience level: Beginner.

Confirmation will be sent to ana@example.com.

Za ovo možete koristiti poseban state:

```
const [submittedData, setSubmittedData] = useState(null);
```

Ako forma ima greške, `submittedData` treba biti `null`.

Korak 13 — Stilizirajte formu

Aplikacija mora biti osnovno stilizirana.

Treba imati:

pozadinsku boju

centralnu karticu

razmake između polja

čitljive labele

stilizirane inpute

stiliziran submit button

crvene poruke greške

zelenu success poruku

Ne mora biti savršen dizajn, ali forma mora izgledati uredno i čitljivo.

Minimalni kriteriji za prolaz

Vježba je uspješno riješena ako:

forma se prikazuje bez grešaka
sva polja su kontrolirana
sva polja koriste formData state
checkbox koristi checked
postoji jedna handleChange funkcija
handleChange radi i za checkbox
postoji errors state
postoji validateField funkcija
postoji validateForm funkcija
validacija radi tijekom promjene polja
validacija radi na submit
forma se ne submitira ako postoje greške
poruke greške se prikazuju ispod polja
nakon ispravnog submitanja prikazuje se success poruka
success poruka prikazuje sažetak prijave

Dodatni izazovi

Izazov 1 — Dodajte phone number

Dodajte polje:

Phone number

Pravila:

nije obavezno

ako je uneseno, mora imati barem 6 znakova

Izazov 2 — Dodajte newsletter checkbox

Dodajte checkbox:

I want to receive workshop updates.

Ovo polje nije obavezno.

Ako je označeno, u success poruci prikažite:

You will receive workshop updates by email.

Ako nije označeno, prikažite:

You will not receive workshop updates.

Izazov 3 — Dodajte Reset button

Dodajte gumb:

Reset form

Klik na gumb treba:

vratiti formData na početne vrijednosti

očistiti errors

očistiti submittedData

Izazov 4 — Dodajte brojač znakova za motivation

Ispod `textarea` polja prikazite:

Characters: X / 30 minimum

Broj X treba se mijenjati dok korisnik tipka.

Izazov 5 — Disable submit button dok uvjeti nisu prihvaćeni

Submit button neka bude disabled dok `acceptedRules` nije označen.

Napomena: i dalje ostavite validaciju za `acceptedRules`, jer korisnik može pokušati submitati na druge načine ili se pravila kasnije mogu promijeniti.

Pitanja za razmišljanje

Na kraju vježbe odgovorite na ova pitanja:

1. Zašto kažemo da je ova forma kontrolirana?
2. Zašto koristimo jedan formData objekt umjesto više useState poziva?
3. Zašto checkbox koristi checked umjesto value?
4. Zašto handleChange treba provjeriti type === "checkbox"?
5. Koja je razlika između validateField i validateForm?
6. Zašto validiramo i tijekom tipkanja i na submit?

7. Zašto koristimo `event.preventDefault()`?
 8. Što bi se dogodilo da ne provjerimo ima li grešaka prije success poruke?
-

Napomena za polaznike

Ova vježba je slična lekciji koju smo zajedno prošli, ali nije ista.

Nemojte samo mehanički kopirati prethodni kod. Prvo razmislite:

Koja polja ima ova forma?
Koje vrijednosti trebaju biti u `formData`?
Koje greške trebaju biti u `errors`?
Koje polje je checkbox?
Koja pravila ima svako polje?
Kada treba prikazati success poruku?

Najvažniji mentalni model ostaje isti:

`formData` čuva vrijednosti forme
`errors` čuva poruke greške
`handleChange` ažurira vrijednost i validira polje
`validateField` provjerava jedno polje
`validateForm` provjerava cijelu formu
`handleSubmit` odlučuje smije li forma biti poslana